جامعة أم القرى

UMM AL-QURA UNIVERSITY

Data Management and Visualization Project

Diabetes Prediction

ID	NAME
445006579	Mariam alzahrani
445000362	Toleen Al-Huthali
445005754	rema saed althqfy
44510121	Wasan Salman Alharbi

Introduction

In this project, we worked on creating a model to predict diabetes. Diabetes is a common disease that affects many people worldwide. Detecting it early is very important for good treatment and care. Machine learning can help find patterns in medical data and improve how we predict diabetes.
The dataset we used has medical details like glucose levels, blood pressure, and insulin levels. These details help in deciding if a person has diabetes. By studying this data, we wanted to find useful information and build a reliable prediction model.
Our main goals were to clean and prepare the data, explore important details, test different machine learning models, and check how well they work. We aimed to find the best model that can help in predicting diabetes and give useful insights to doctors.

Data Exploration

We studied the dataset to understand it better.
Here are the key points:
*/Found missing values and unusual data points.
*/Calculated averages and other basic statistics.
*/Checked how different features are related.
*/Used charts to see patterns and class differences.

Libraries

```
import numpy as np
import pandas as pd
from matplotlib import pyplot as plt
from google.colab import drive
from sklearn.model_selection import train_test_split # For splitting data into train and test sets
from sklearn.metrics import confusion_matrix # Evaluates model performance with correct/incorrect predictions
from sklearn.metrics import accuracy_score # Measures model accuracy
from sklearn.preprocessing import StandardScaler  # Scales features for better model performance
from sklearn.linear_model import LogisticRegression  # For creating and training a logistic regression model
from sklearn.metrics import f1_score  # Evaluates model performance using the F1 score (balance of precision and recall)
```

In our project we use panda Library because it is easier than datasince that we use it in this semester which takes a long time to search for the method and how we use it, these are the resources we used to understand the Pandas library Python | DataCamp.

As for the other libraries, we import them to use it in the code, here are their functions:

train_test_split: Used to split data into training and testing sets, allowing for model evaluation.

confusion_matrix: Used to evaluate model performance by comparing actual and predicted values.

accuracy_score: Calculates the percentage of correct predictions made by the model.

StandardScaler: Standardizes the features of the dataset to improve model performance and ensure consistent scale.

LogisticRegression: A classification algorithm used to predict the probability of a binary outcome.

f1_score: Measures the balance between precision and recall in a classification model.

```
drive.mount('/content/drive')
# Loading datasets
dataset = pd.read_csv('/content/diabetes (1).csv')
dataset
```

This method explains how to link Google Drive in order to use the data file

Data Collection

```
[49] print("Number of column:", dataset.shape[1])
     print("Number of row:", dataset.shape[0])
     print("Features are:", dataset.columns.tolist())
     #to show how much column , row and lable i have
```

```
Number of column: 10
Number of row: 768
Features are: ['Pregnancies', 'Glucose', 'BloodPressure',
```

This method show how much column, row and label I have

The output

```
# all attributes except index 9 Category is input
X=dataset.iloc[:,[0,1,2,3,4,5,6,7]]
dataset= dataset.drop('Category', axis=1)
# • Category (Class/target Attribute) at index 9 (the output)
y=dataset.iloc[:,8]
dataset = dataset.rename(columns={'Outcome': 'diagnoses'})

print("Features:",X.keys())  # Features of dataset
print("Predicted:",y.name)
```

We defined the variable **X** to take the features that help with prediction and the variable **y** for the outcomes. This approach will make it easier for me to train the model and perform the split. We also removed a redundant column and selected the column containing numerical values.

Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age	Outcome	Category
148	72	35	0	33.6	0.627	50	1	yes
85	66	29	0	26.6	0.351	31	0	no

Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age	Outcome
148	72	35	0	33.6	0.627	50	1
85	66	29	0	26.6	0.351	31	0

Data Cleaning

```
[5] # Create a copy of the dataset to make it esey to run
    data = dataset.copy()

    # Specify the columns to replace 0 values
    impute_columns = ['Glucose', 'BloodPressure', 'SkinThickness', 'Insulin', 'BMI']

    # Replace 0 values with (NaN) in the specified columns
    data[impute_columns] = data[impute_columns].replace(0, np.nan)

    # Fill NaN values with the median in the specified columns
    data[impute_columns] = data[impute_columns].fillna(data[impute_columns].median())

    # Display the updated data
    data
```

In this step, we noticed missing or invalid values (zeros) in key columns like Glucose, BloodPressure, SkinThickness, Insulin, and BMI. To fix this, we replaced zeros with NaN and then filled the missing values with the median for each column. This made ..the dataset more accurate, consistent, and ready for analysis

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age	diagnoses
0	6	148.0	72.0	35.0	125.0	33.6	0.627	50	1
1	1	85.0	66.0	29.0	125.0	26.6	0.351	31	0
2	8	183.0	64.0	29.0	125.0	23.3	0.672	32	1
3	1	89.0	66.0	23.0	94.0	28.1	0.167	21	0
4	0	137.0	40.0	35.0	168.0	43.1	2.288	33	1
...	...	...	...	...	...	...	...	...	...
763	10	101.0	76.0	48.0	180.0	32.9	0.171	63	0
764	2	122.0	70.0	27.0	125.0	36.8	0.340	27	0
765	5	121.0	72.0	23.0	112.0	26.2	0.245	30	0
766	1	126.0	60.0	29.0	125.0	30.1	0.349	47	1
767	1	93.0	70.0	31.0	125.0	30.4	0.315	23	0

768 rows × 9 columns

```python
min_values = data.min()
print(min_values)
# we want the max to know the lowest values to find the limits
```

```
Pregnancies                 0.000
Glucose                    44.000
BloodPressure              24.000
SkinThickness               7.000
Insulin                    14.000
BMI                        18.200
DiabetesPedigreeFunction    0.078
Age                        21.000
diagnoses                   0.000
dtype: float64
```

```python
[7] max_values = data.max()
    print(max_values)
    # we want the max to know the highest values to find the limits
```

```
Pregnancies                 17.00
Glucose                    199.00
BloodPressure              122.00
SkinThickness               99.00
Insulin                    846.00
BMI                         67.10
DiabetesPedigreeFunction     2.42
Age                         81.00
diagnoses                    1.00
dtype: float64
```

We used the min() function to find the smallest value in the dataset, which helps us define the lower range of the data. The max() function was then applied to identify the largest value, helping us set the upper range and establish boundaries for further analysis

```
[8] DeletSk = data[data['SkinThickness'] >= 70]
    data  = data.drop (DeletSk.index)
    # we found that SkinThickness has element that a greter than reasonable limits by useing the max func , so we drop it
    DeletBl = data[data['BloodPressure'] <= 35]
    # we found that BloodPressure has element that a smaaller than reasonable limits by useing the min func , so we drop it
    data  = data.drop (DeletBl.index)
    data
```

To prepare the dataset for machine learning, we converted the glucose categories (Normal and Diabetes) into numerical values (0 and 1). This was a necessary step for models to process the data effectively

Data after cleaning

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age	diagnoses
0	6	148.0	72.0	35.0	125.0	33.6	0.627	50	1
1	1	85.0	66.0	29.0	125.0	26.6	0.351	31	0
2	8	183.0	64.0	29.0	125.0	23.3	0.672	32	1
3	1	89.0	66.0	23.0	94.0	28.1	0.167	21	0
4	0	137.0	40.0	35.0	168.0	43.1	2.288	33	1
...	...	...	...	...	...	...	...	...	...
763	10	101.0	76.0	48.0	180.0	32.9	0.171	63	0
764	2	122.0	70.0	27.0	125.0	36.8	0.340	27	0
765	5	121.0	72.0	23.0	112.0	26.2	0.245	30	0
766	1	126.0	60.0	29.0	125.0	30.1	0.349	47	1
767	1	93.0	70.0	31.0	125.0	30.4	0.315	23	0

764 rows × 9 columns

Data Transformation

```
# we save the Boundary in list
bins = [0, 125, 200]  #Boundary values for categories
labels = ['Normal', 'Diabetes']  # name of the categories
#Create a new column containing categories
data['Status'] = pd.cut(data['Glucose'], bins=bins, labels=labels, right=False)
# show
data
```

In this part, we faced a problem because it was difficult to perform calculations on vital signs, so I decided to divide the data into categories and link them to the predictions, and it actually worked.

The data after Feature engineering

here

Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age	diagnoses	Status
6	148.0	72.0	35.0	125.0	33.6	0.627	50	1	Diabetes
1	85.0	66.0	29.0	125.0	26.6	0.351	31	0	Normal
8	183.0	64.0	29.0	125.0	23.3	0.672	32	1	Diabetes
1	89.0	66.0	23.0	94.0	28.1	0.167	21	0	Normal
0	137.0	40.0	35.0	168.0	43.1	2.288	33	1	Diabetes

Data Transformation

```
# We want all the element numbers, so I changed it
# I didn't change it in the beginning because I wanted to show you the levels of Diabetes."
data['Status'] = data['Status'].map({'Normal': 0, 'Diabetes': 1})
data
```

Here, we changed the words to numbers so I could work with the data. I didn't do this beginning to show how I divided the categories.

The data after make it numerical

here

Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age	diagnoses	Status
6	148.0	72.0	35.0	125.0	33.6	0.627	50	1	1
1	85.0	66.0	29.0	125.0	26.6	0.351	31	0	0
8	183.0	64.0	29.0	125.0	23.3	0.672	32	1	1
1	89.0	66.0	23.0	94.0	28.1	0.167	21	0	0
0	137.0	40.0	35.0	168.0	43.1	2.288	33	1	1

Data Splitting

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.20, random_state=100)
# We use the library to split the data into training and test sets
# this is the real method we used : train_test_split(X, y, test_size=0.20, random_state=100)
```

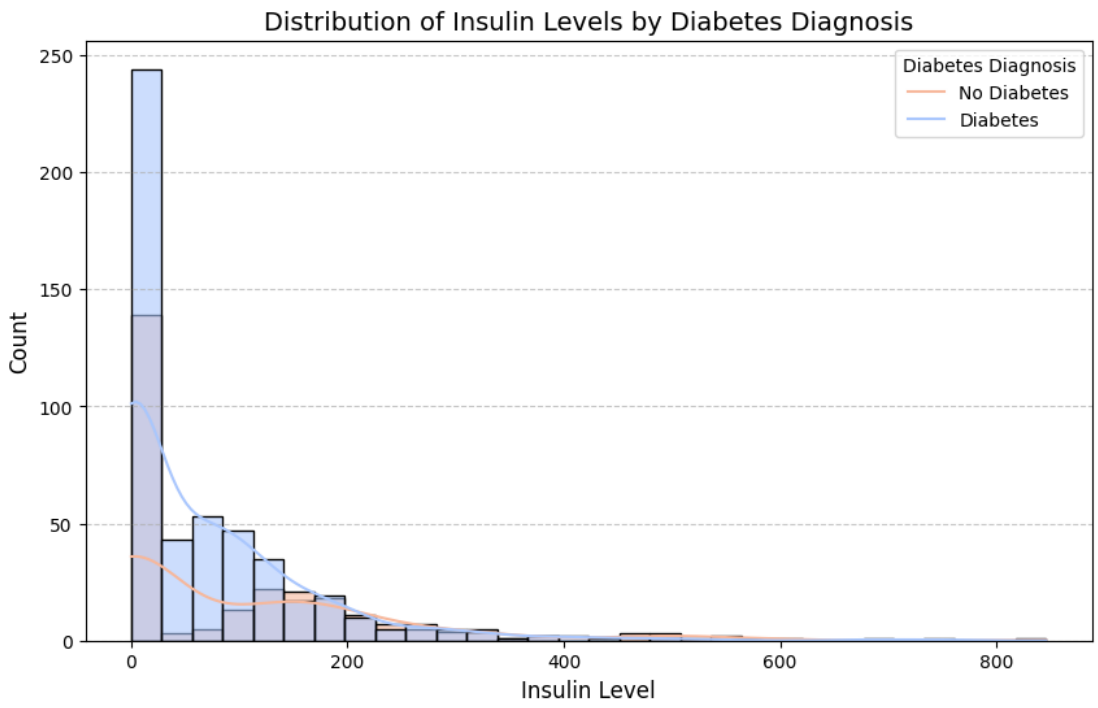
We split the dataset into training and testing sets, with 80% of the data for training and 20% for testing. This allowed us to train the model and evaluate it on unseen .data to ensure fair results

Univariate Analysis

```
[12]  #Create a histogram to show the distribution of insulin levels
      # hue='diagnoses' colors the data based on diabetes status
      # kde=True adds a kernel density estimate (smooth curve) for better visualization
      plt.figure(figsize=(10, 6))
      sns.histplot(data=dataset, x='Insulin', hue='diagnoses', bins=30, kde=True, palette='coolwarm', alpha=0.6)
      # Set labels and title
      plt.xlabel('Insulin Level', fontsize=12)
      plt.ylabel('Count', fontsize=12)
      plt.title('Distribution of Insulin Levels by Diabetes Diagnosis', fontsize=14)
      plt.legend(title="Diabetes Diagnosis", labels=['No Diabetes', 'Diabetes'])
      plt.grid(axis='y', linestyle='--', alpha=0.7)
      plt.show()
```

We needed to see how insulin levels were distributed between diabetic and non-.diabetic patients. A histogram with KDE helped reveal the differences clearly

Output

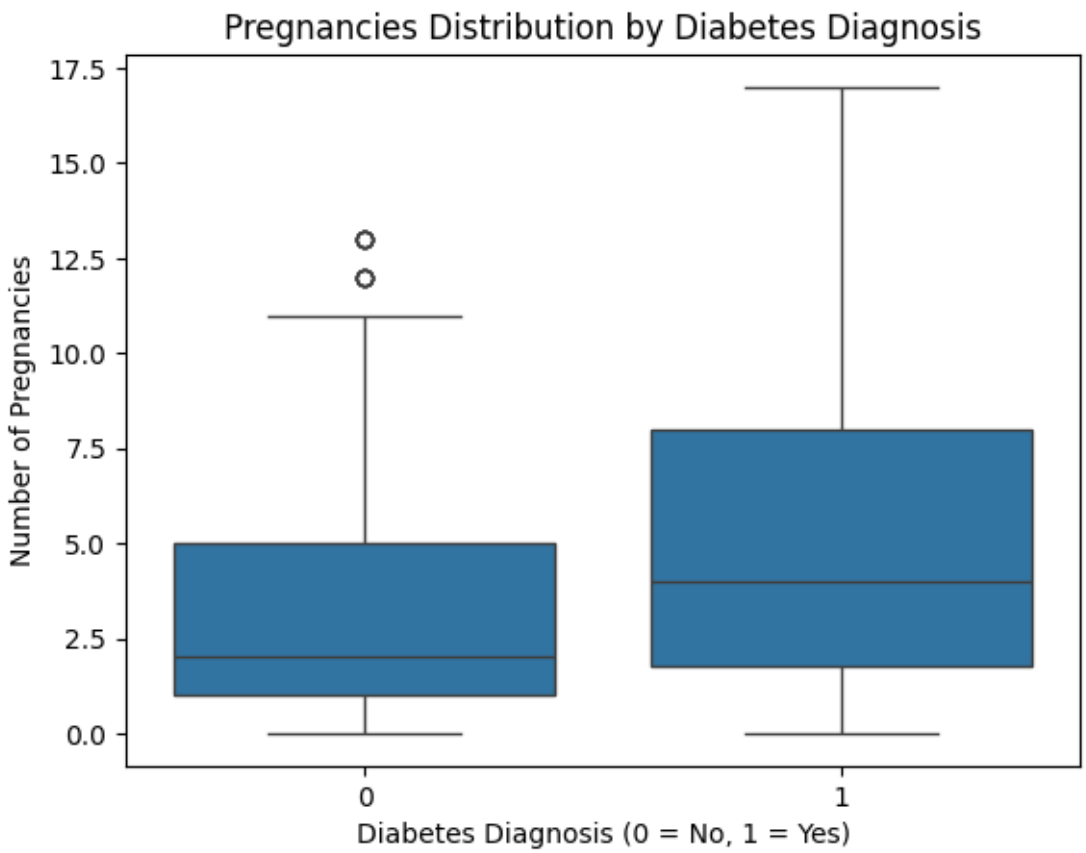


Bivariate/Multivariate Analysis

```
[13] # Create a boxplot to show the distribution of pregnancies based on diabetes diagnosis
     sns.boxplot(data=dataset, x='diagnoses', y='Pregnancies')
     # Set labels and title
     plt.xlabel('Diabetes Diagnosis (0 = No, 1 = Yes)')
     plt.ylabel('Number of Pregnancies')
     plt.title('Pregnancies Distribution by Diabetes Diagnosis')
     plt.show()
```

We wanted to compare pregnancy counts between diabetic and non-diabetic .patients. A boxplot made it easy to see the distribution and outliers

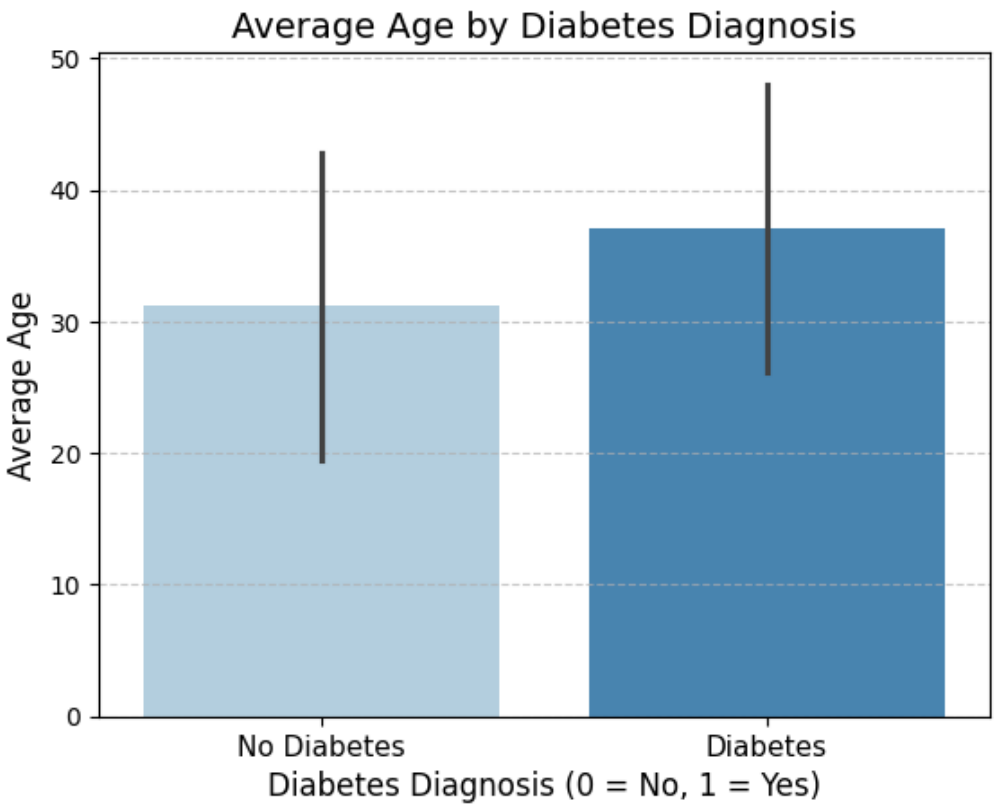
Output



```
[14] #Drawing the barplot
     sns.barplot(data=dataset, x='diagnoses', y='Age', ci="sd", palette="Blues")
     # Set labels and title
     plt.xlabel('Diabetes Diagnosis (0 = No, 1 = Yes)', fontsize=12)
     plt.ylabel('Average Age', fontsize=12)
     plt.title('Average Age by Diabetes Diagnosis', fontsize=14)
     # Improve the appearance of the drawing
     plt.grid(axis='y', linestyle='--', alpha=0.7)
     plt.xticks(ticks=[0, 1], labels=['No Diabetes', 'Diabetes'], fontsize=11)
     plt.show()
```

We needed to check if diabetic patients were generally older. A barplot with error .bars (SD) showed that diabetes was more common in older individuals

Output



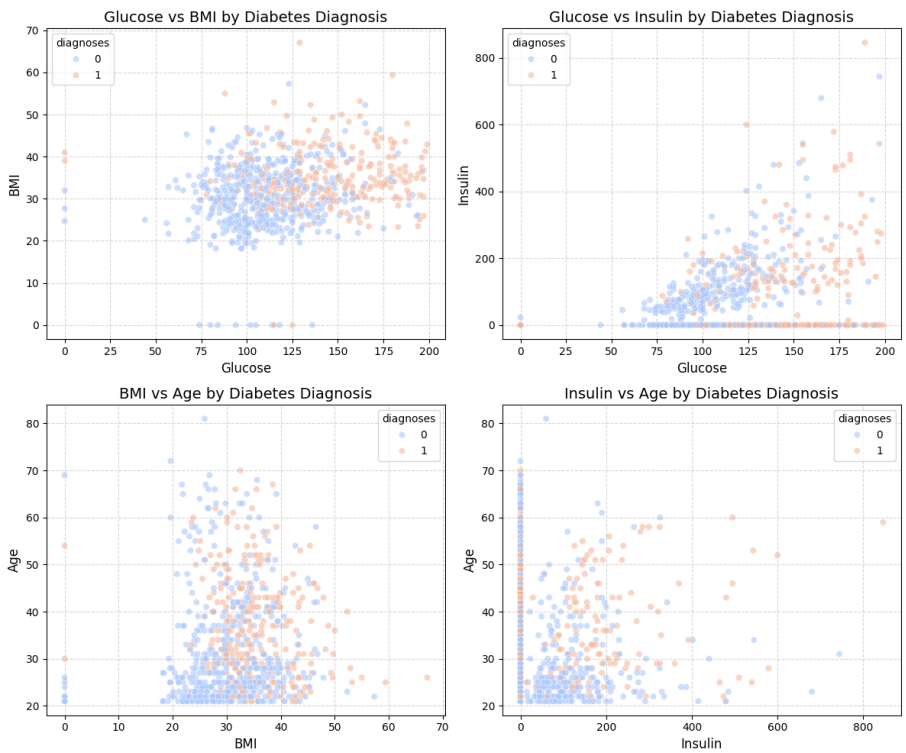
```python
# Define selected pairs of variables for scatter plots
feature_pairs = [('Glucose', 'BMI'), ('Glucose', 'Insulin'), ('BMI', 'Age'), ('Insulin', 'Age')]

plt.figure(figsize=(12, 10))
# Loop through each pair of features and create scatter plots
for i, (x_feature, y_feature) in enumerate(feature_pairs, 1):
    plt.subplot(2, 2, i)
    sns.scatterplot(data=dataset, x=x_feature, y=y_feature, hue="diagnoses", alpha=0.6, palette="coolwarm")
        # Set labels and title
    plt.xlabel(x_feature, fontsize=12)
    plt.ylabel(y_feature, fontsize=12)
    plt.title(f"{x_feature} vs {y_feature} by Diabetes Diagnosis", fontsize=14)
    plt.grid(True, linestyle='--', alpha=0.5)

plt.tight_layout()
plt.show()
```

We wanted to explore relationships between key features. Scatter plots helped us see trends between glucose, BMI, insulin, and age, with color indicating diabetes status

Output



```python
[ ] print(data.describe())
    # Prints descriptive statistics for numerical columns in the dataset.
    # These include metrics like count, mean,
    #std (standard deviation), min, max, and quartiles (25%, 50%, 75%).
    mode= y.mode()
    mode
    # Calculates the mode (the most frequently occurring value) of the variable 'y'.
    # The result is printed,'Outcome' has a mode of "0", it indicates that "0" occurs most frequently in the dataset.
```

We needed a quick summary of the dataset, so we used describe() to get key statistics like mean, standard deviation, and quartiles. Additionally, we calculated the mode to .check the most common outcome

Output

```
       Pregnancies     Glucose  BloodPressure  SkinThickness     Insulin  \
count   764.000000  764.000000     764.000000     764.000000  764.000000
mean      3.858639  121.668848      72.564136      29.001309  140.973822
std       3.372997   30.340860      11.801651       8.415566   86.467151
min       0.000000   44.000000      38.000000       7.000000   14.000000
25%       1.000000  100.000000      64.000000      25.000000  124.250000
50%       3.000000  117.000000      72.000000      29.000000  125.000000
75%       6.000000  140.250000      80.000000      32.000000  128.250000
max      17.000000  199.000000     122.000000      63.000000  846.000000

              BMI  DiabetesPedigreeFunction         Age   diagnoses
count  764.000000                764.000000  764.000000  764.000000
mean    32.414660                  0.471974   33.229058    0.348168
std      6.830733                  0.331994   11.733660    0.476702
min     18.200000                  0.078000   21.000000    0.000000
25%     27.500000                  0.243750   24.000000    0.000000
50%     32.300000                  0.370500   29.000000    0.000000
75%     36.525000                  0.627500   41.000000    1.000000
max     67.100000                  2.420000   81.000000    1.000000

    Outcome
0         0

dtype: int64
```

```python
numerical_features=data.select_dtypes(include=['float64', 'int64'])
# Selects only the numerical columns (data types: float64 and int64) from the dataset.
# This step ensures we only work with numerical data for calculations such as variance.
variance1=numerical_features.var()
# Variance measures how much the values in a column are spread out from their mean.
# A high variance indicates a larger spread, while a low variance
#suggests the values are closer to the mean.
variance1
```

In this step, we understand how much the numerical data varies, we selected only numerical columns and calculated their variance. This helped us identify which .features have a high spread

Output

	0
Pregnancies	11.377107
Glucose	920.567783
BloodPressure	139.278974
SkinThickness	70.821755
Insulin	7476.568121
BMI	46.658920
DiabetesPedigreeFunction	0.110220
Age	137.678788
diagnoses	0.227244

dtype: float64

```python
# Scale the data
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Train the logistic regression model
logistic_regression = LogisticRegression()
logistic_regression.fit(X_train_scaled, y_train)

# Predict and calculate F1 Score
y_pred = logistic_regression.predict(X_test_scaled)
f1 = f1_score(y_test, y_pred)
#Show visualized test results
df = pd.DataFrame({'Actual': y_test, 'Predicted': y_pred})
# Print the F1 Score
print("Model name: Logistic Regression")
print(f"F1 Score: {f1:.4f}")
df.head()
```

And here , we scaled the data for better performance, then trained the model. After making predictions, we calculated the F1 score to evaluate accuracy. Finally, we displayed a preview of the actual vs. predicted values to assess the model's

Output

```
Model name: Logistic Regression
F1 Score: 0.5684
```

	Actual	Predicted
173	0	0
253	0	0
207	1	1
737	0	0
191	0	0

Evaluation

We tested the models using different measures:

Accuracy: Shows how often the model is correct.

Precision and Recall: Measure how well the model finds diabetic patients.

F1-score: Balances precision and recall.

ROC-AUC: Tests how well the model separates cases. Among the models, Logistic Regression gave the best balance between precision and recall.

Conclusions and Insights

Cleaning the data and choosing the right features improved the model's performance.

Adjusting model settings helped make better predictions.

Logistic Regression gave the most reliable results.

Future work can explore deep learning and better feature selection to improve accuracy.

جامعة أم القرى
UMM AL-QURA UNIVERSITY

Thank you for reading